

PARALLEL AND CONCURRENT PROGRAMMING

Implementing the two locking algorithms, Filter Lock and Bakery Lock and comparing the performance of these locks by measuring the average waiting time for threads to obtain the locks and the throughput.

1. Filter Lock :

Filter Lock algorithm is used in concurrent programming to allow multiple threads to compete for a shared resource by allowing only one thread to enter the critical section. Filter Lock algorithm ensures mutual exclusion among threads trying to enter the critical section.

In filterlock every thread pass through several levels before entering the critical section and only one thread makes it to the final level to acquire the lock and enter the critical section.

Algorithm Overview-

The number of levels is equal to the number of threads ranging from 0 to N-1, where N is the number of threads present.

Levels[i] indicates the current level of thread i from 0 to N-1.

Victim[j] indicates that which thread is the victim at level j.

The thread i enters the lock by trying to move from level 0 to the highest level present to ultimately get the critical section.

At every level j , the thread sets its level to j and declares itself as the victim for that level.

The thread waits at a particular level till every thread at that level either drops or moves to higher level.

Once the thread finishes its work at the critical section it sets its level back to 0, signaling that it has completed its work and other threads can proceed.

Properties of filterlock algorithm-

Filterlock satisfies the property of -

Mutual Exclusion

Bounded Waiting and

Progress

It is Starvation Free and

Deadlock Free.

2. Bakery Lock :

Bakery lock algorithm is a mutual exclusion satisfying algorithm used in concurrent programming to ensure that only one thread enters into the critical section at any point of time.

In bakerylock every thread has a number and enters the critical section only if its number is less than the number of other threads trying to enter critical section.

Algorithm Overview-

Each thread picks up a number greater than 0 and greater than number of any other thread present which represents its turn to enter the critical section.

After selecting a number, the thread waits until it has the lowest number compared to all other threads present who want to enter the critical section by continuously checking whether other threads have chosen smaller number.

once a thread acquires a smaller number it enters the critical section.

After completing its work in critical section, the thread sets its number back to zero signaling that it no longer wants to enter the critical section.

Properties of bakerylock algorithm-

Bakerylock satisfies the property of-

Mutual Exclusion

Bounded Waiting

Deadlock Free

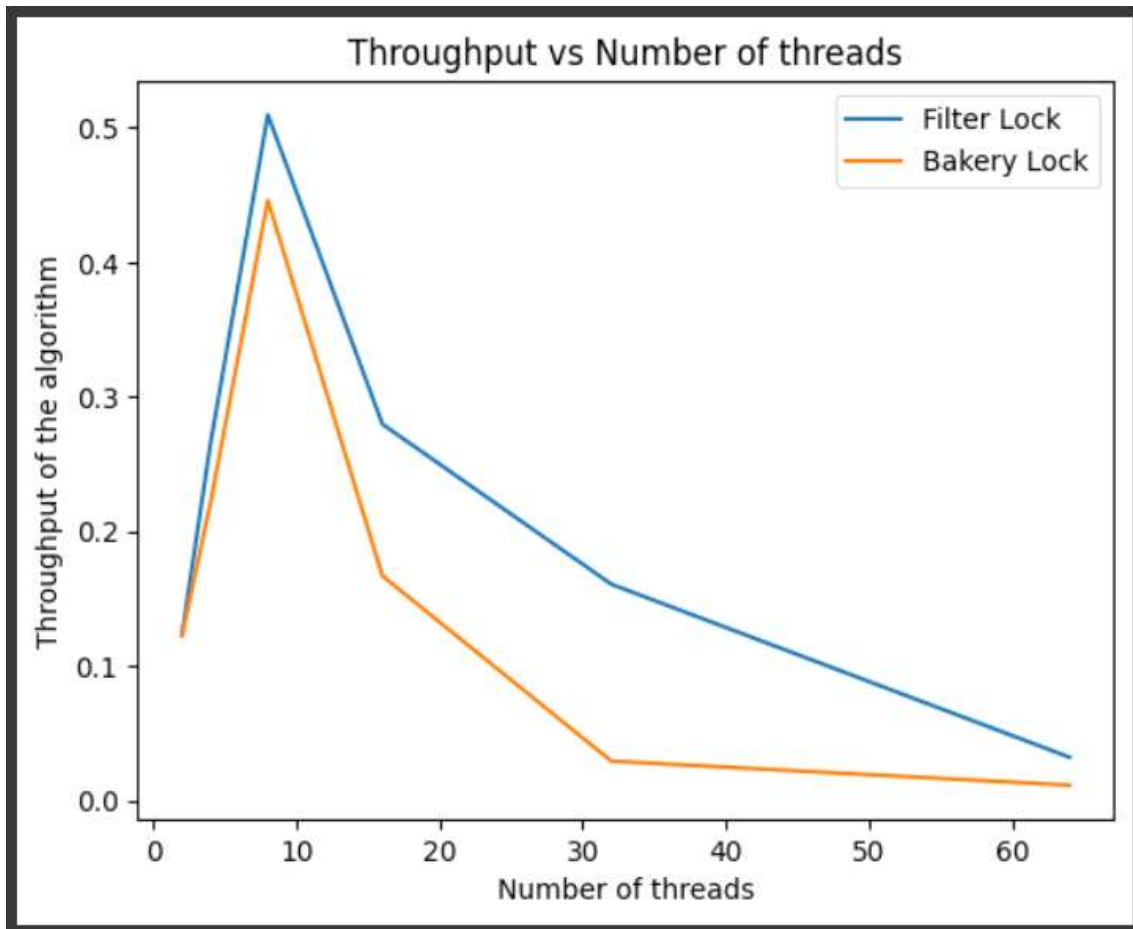
Starvation Free

Graphs:

To measure the performance of both the algorithms we will create plots depending on different varying measures :

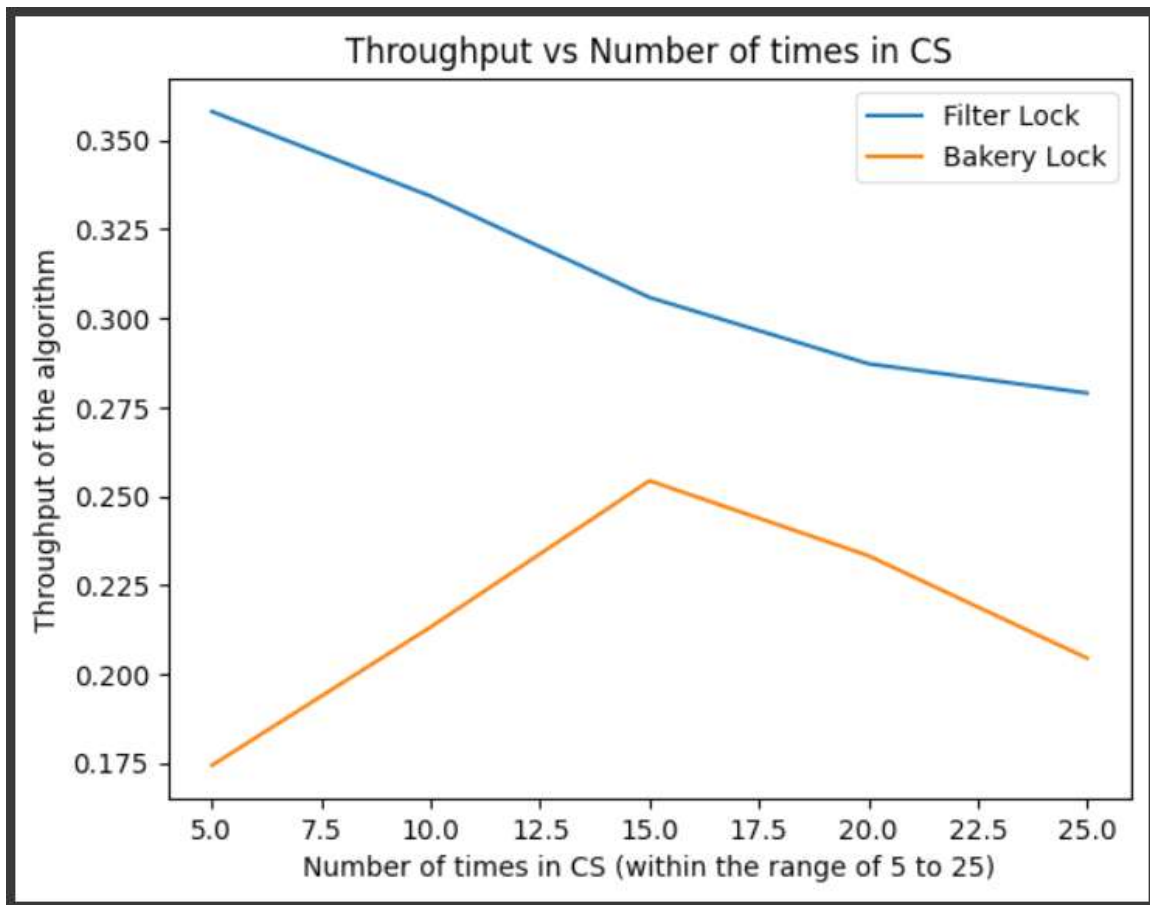
1. Throughput vs Number of threads :

As we go on increasing the number of threads, throughput of the algorithm increases till a certain point but after that if we increase number of threads, the throughput of the algorithm falls drastically.



2. Throughput vs Number of times in critical section :

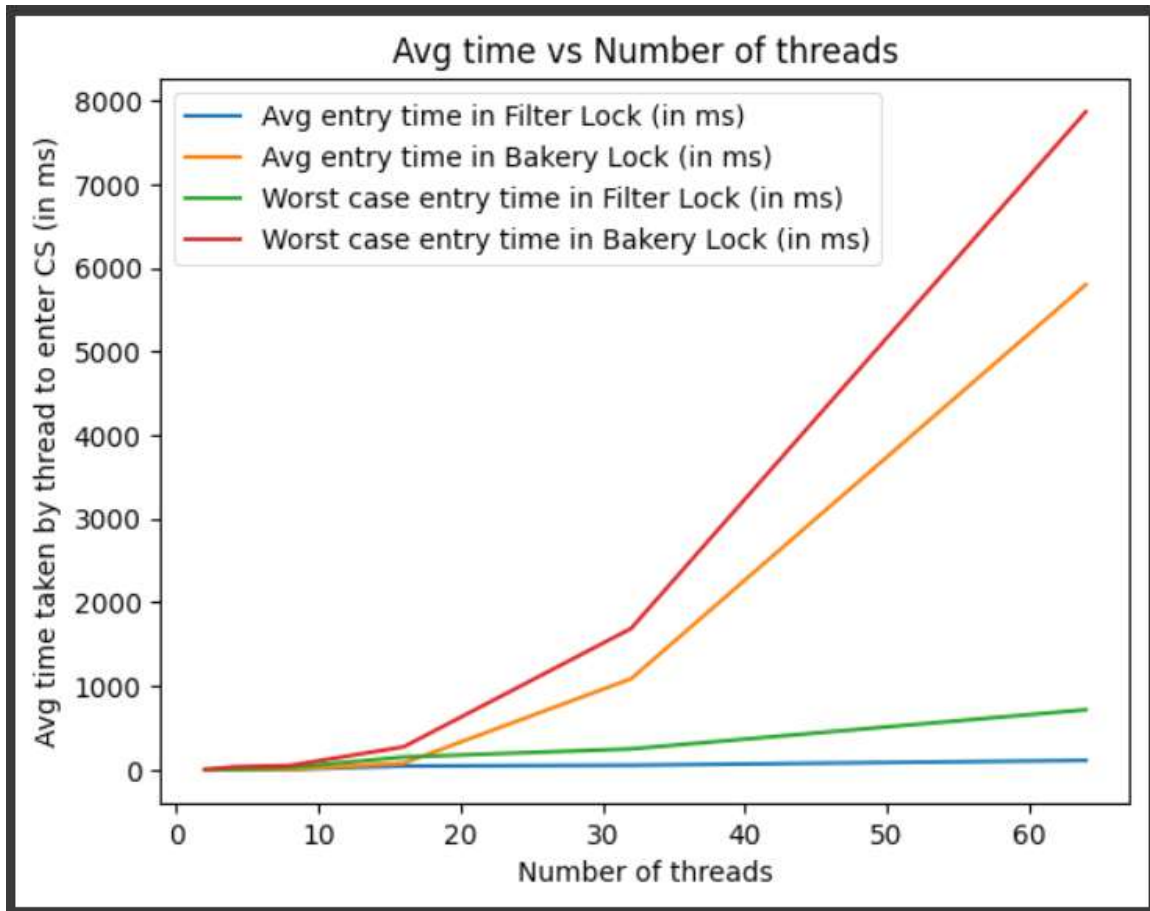
On increasing the number of times a thread wants to enter the critical section, the throughput of the system decreases.



3. Avg time vs Number of threads :

On increasing the number of threads for comparing the average time required by threads to enter the critical section or the worst case time in both the algorithms also increases.

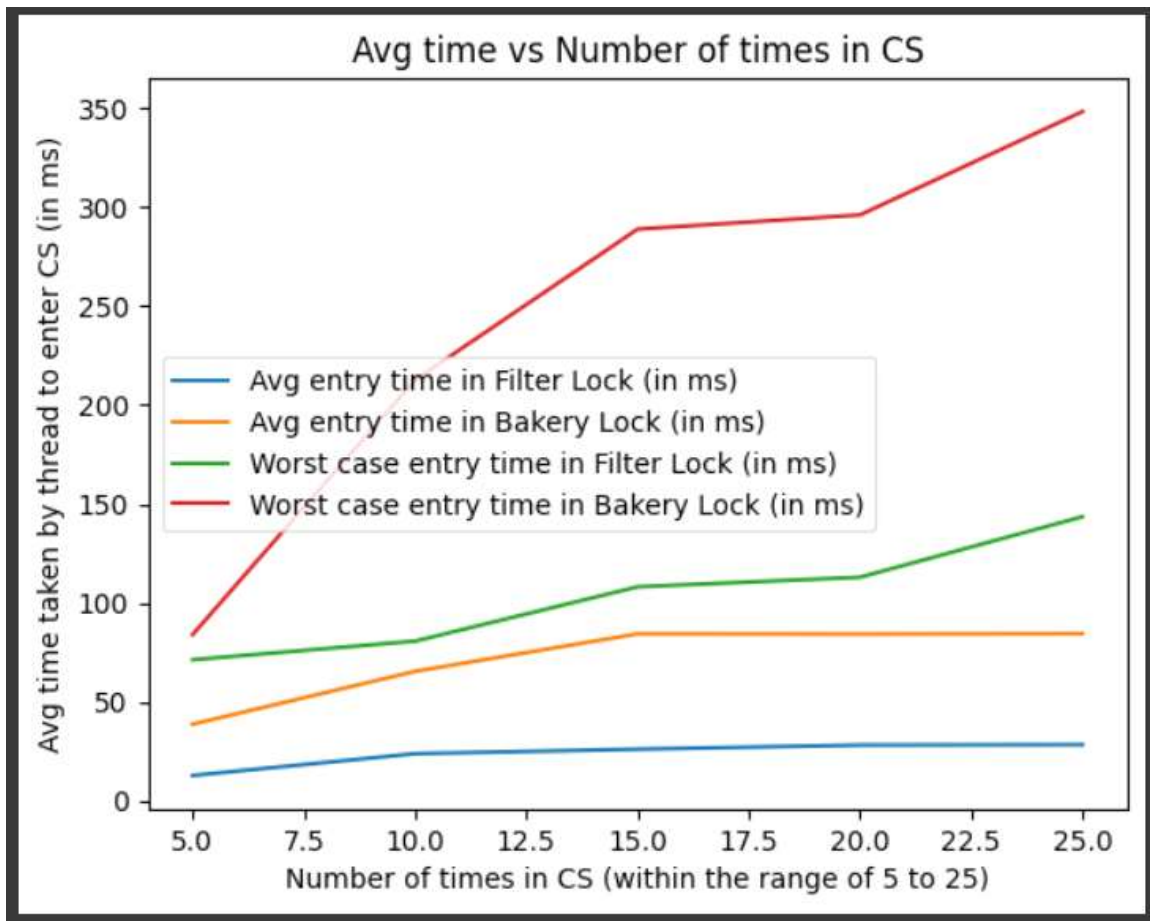
Also it is clearly visible that on increasing the number of threads in the case of bakery lock algorithm the average time taken increases exponentially making the algorithm less efficient.



4. Avg Time vs Number of times in critical section :

On increasing the number of times a thread wants to enter the critical section for comparing the average time required by threads to enter the critical section or the worst case time in both the algorithms also increases. Also we can see that on increasing the number of times a thread wants to enter the critical section in the case of bakery lock algorithm the average

time taken increases drastically making the algorithm less efficient.



Conclusion:

Seeing the above scenarios of both the algorithm filterlock and bakerylock, for comparing their performance on the basis of average time required to acquire the lock and the throughput we have seen that although both the algorithms show similar behaviour on different varying parameters but bakery lock algorithm takes more time at every aspect as compared to filterlock algorithm.

Whether it be changing the number of threads or the number of times

threads wants to enter the critical section or the avg time required by thread to get critical section or even the worst case time, it is clearly visible that the filterlock algorithm is taking more time as compared to bakery lock algorithm.